# DigiTrainer

## *Benutzer- und Experimentierhandbuch*

Manfred Becker

Version 0.69, 2026-11-02

# Inhaltsverzeichnis


1. Vorwort . . . 2
2. Aufbau dieses Handbuchs . . . 3
   2.1. Geplante Kapitel . . . 3
3. Teil I: Den DigiTrainer bedienen . . . 4
   3.1. Erster Start . . . 4
   3.2. Bedienelemente . . . 4
      3.2.1. Hauptschalter . . . 4
      3.2.2. Schiebeschalter und LEDs . . . 4
      3.2.3. Steckbuchsen . . . 4
   3.3. Patchkabel bearbeiten . . . 4
      3.3.1. Leitung stecken . . . 4
      3.3.2. Leitung entfernen . . . 5
      3.3.3. Leitung umstecken . . . 5
      3.3.4. Rückgängig . . . 5
   3.4. Spannungsversorgung und Verbindungsschienen . . . 5
   3.5. Anzeige, Zoom und Touch . . . 5
   3.6. Vermessungsmodus . . . 6
   3.7. Die Schaltplanansicht (neu in v0.69) . . . 6
      3.7.1. Ein Schaltbild für das Handbuch exportieren . . . 6
4. Teil II: Digitale Elektronik und simulierte Baugruppen . . . 7
   4.1. Grundlagen: LOW, HIGH und Wahrheitstabellen . . . 7
      4.1.1. NOT (Inverter) . . . 7
      4.1.2. AND . . . 7
      4.1.3. OR . . . 7
      4.1.4. XOR . . . 7
   4.2. Die sechs Logik-IC-Sockel . . . 8
   4.3. Taktgenerator . . . 8
   4.4. 4-Bit-Binärzähler SN74LS293N . . . 8
   4.5. 7-Segment-Anzeige MC14495P . . . 8
   4.6. 4-Bit-ALU SN74LS382N . . . 8
   4.7. 6-Bit-A/D-Wandler NE5037N . . . 9
5. Teil III: Experimente . . . 10
   5.1. So sind die Experimente aufgebaut . . . 10
   5.2. Experiment 1 – Schalter, LED und NOT-Gatter . . . 10
      5.2.1. Lernziele . . . 10
      5.2.2. Vorbereitung . . . 10
      5.2.3. Experiment 1a – Schalter und LED . . . 10
      5.2.4. Experiment 1b – Das NOT-Gatter (Inverter) . . . 11

5.2.5. Vergleich der beiden Versuche . . . 12
5.2.6. Fragen und Übungen . . . 13
5.2.7. Merksatz . . . 13
5.3. Experiment 2: Ein 4-Bit-Zähler zeigt Hexadezimalzahlen . . . 13
5.4. Experiment 3: Binärzähler und LED-Ausgänge . . . 13
5.5. Experiment 4: Lauflicht D0 bis D7 . . . 14
5.6. Experiment 5: Pendellauflicht . . . 14
6. Teil IV: Projekte, Fehlersuche und Technik . . . 15
6.1. Projekte speichern und laden . . . 15
6.2. Beschreibungen in JSON . . . 15
6.3. Eigene Projekte beitragen . . . 15
6.4. Debug-Anzeige . . . 15
6.5. Typische Fehler . . . 16
6.6. Technische Umsetzung und Grenzen . . . 16
6.7. Wie entsteht der automatische Schaltplan? . . . 16
Anhang A: Funktions- und Quellenprüfung . . . 18
Anhang B: Quellen und Mitwirkende . . . 19
Anhang C: IC-Katalog für künftige Experimente . . . 20
C.1. Bereits simulierte ICs . . . 20
C.2. Logikgatter und kombinatorische Funktionen . . . 20
C.3. Flipflops und Schieberegister . . . 21
C.4. Zähler und Frequenzteiler . . . 21
C.5. Decoder, Multiplexer und Anzeige . . . 22
C.6. Arithmetik und Vergleicher . . . 22
C.7. Besonders interessante 20-polige Bausteine . . . 22
C.8. Sockel und elektrische Besonderheiten . . . 23
C.9. Vorgeschlagene Reihenfolge für die IC-Bibliothek . . . 23

## Arbeitsfassung 0.3 – Benutzerteil, Schaltplanerstellung, erste Experimente und IC-Katalog.



Dieses Handbuch beschreibt den dokumentierten Funktionsstand von DigiTrainer v0.69. Die Bedienfunktionen und Hardwaremerkmale basieren auf der Projekt-README, der Online-Hilfe und den beschriebenen Neuerungen von Pull Request #4. Die konkreten Steckbuchsenverbindungen der späteren Experimente werden vor der Veröffentlichung anhand von `board.json` und den gespeicherten Projektdateien verifiziert. Ungeprüfte Verdrahtungslisten werden bewusst nicht als fertige Bauanleitung ausgegeben.

# Kapitel 1. Vorwort

Der ITT Digi-Trainer 2 ist ein Lehr- und Experimentiersystem für digitale Elektronik. Der browserbasierte DigiTrainer bildet das Gerät anhand eines Fotos und interaktiver Bedienelemente nach. Statt physische Patchkabel zu stecken, verbindet man Buchsen mit Maus oder Finger. Der Simulator ermittelt die elektrischen Netze und berechnet die logischen Zustände der simulierten Baugruppen.

Das Handbuch richtet sich an Neugierige ohne Vorkenntnisse ebenso wie an Menschen, die bereits mit TTL-Bausteinen gearbeitet haben. Es verbindet die Bedienung des Programms mit nachvollziehbaren Experimenten. Eine Installation ist für die Online-Version nicht erforderlich.

**Live-Demo:** https://manibecker.github.io/DigiTrainer/

**Quellcode:** https://github.com/ManiBecker/DigiTrainer

# Kapitel 2. Aufbau dieses Handbuchs

Teil I führt in die Bedienung ein. Teil II erklärt die digitale Elektronik und die simulierte Hardware. Teil III enthält praktische Experimente. Teil IV behandelt Projekte, Fehlersuche, die automatische Schaltplanerstellung und technische Hintergründe.

## 2.1. Geplante Kapitel

1. Einführung und erster Start
2. Die Oberfläche und ihre Bedienelemente
3. Patchkabel, Versorgung und Verbindungsschienen
4. Anzeige, Zoom, Vollbild und Touch
5. Grundlagen der Digitaltechnik
6. Logikgatter und die sechs IC-Sockel
7. Taktgenerator und 4-Bit-Binärzähler
8. 7-Segment-Anzeige und 4-Bit-ALU
9. Experimenteller A/D-Wandler
10. Projekte speichern, laden und teilen
11. Debug-Anzeige und Fehlersuche
12. Automatische Schaltplanerstellung und SVG-Export
13. Experimente: vom Inverter zum Lauflicht
14. Technische Umsetzung und Grenzen
15. Anhang: Anschlussbelegungen, Wahrheitstabellen, JSON-Format

# Kapitel 3. Teil I: Den DigiTrainer bedienen

## 3.1. Erster Start

1. Öffne https://manibecker.github.io/DigiTrainer/ in einem modernen Browser.
2. Warte, bis das Foto des Digi-Trainer 2 vollständig angezeigt wird.
3. Lasse den Hauptschalter zunächst auf **OFF**. In dieser Stellung darfst du die Schaltung verdrahten.
4. Schalte erst nach Abschluss der Verdrahtung auf **ON**. Die rote Betriebsanzeige signalisiert den eingeschalteten Zustand.
5. Betätige die Schiebeschalter S0 bis S7 oder den Taktgenerator und beobachte die Ausgangs-LEDs.



Zum Kennenlernen eignet sich ein fertiges Beispielprojekt. Öffne **Projekt**, wähle unter **Beispielprojekte** einen Binärzähler aus, lies die Beschreibung, lade das Projekt und schalte den DigiTrainer ein.

## 3.2. Bedienelemente

### 3.2.1. Hauptschalter

Der Hauptschalter trennt den Verdrahtungsmodus vom Simulationsbetrieb. Im ausgeschalteten Zustand lassen sich Leitungen stecken und bearbeiten. Im eingeschalteten Zustand kann die Schaltung mit Schaltern und Taktgenerator betrieben werden. Ein fotografisches Overlay und eine rote Betriebsanzeige verdeutlichen den Zustand.

### 3.2.2. Schiebeschalter und LEDs

Acht Schiebeschalter S0 bis S7 liefern manuell einstellbare logische Signale. Acht Ausgangs-LEDs D0 bis D7 machen Ergebnisse sichtbar. In der Debug-Anzeige erscheinen zusätzlich die binären Werte 0 und 1.

### 3.2.3. Steckbuchsen

Mehr als 200 definierte Hotspots repräsentieren Buchsen und Anschlüsse des Originalgeräts. Beim Zeigen auf Anschlüsse können deren Bezeichnungen und IC-Pinfunktionen angezeigt werden. Für die Dokumentation verwenden wir die tatsächlichen Anschlussnamen des Simulators.

## 3.3. Patchkabel bearbeiten

### 3.3.1. Leitung stecken

1. DigiTrainer ausschalten.
2. Eine Steckbuchse mit der linken Maustaste oder dem Finger berühren.

3. Zur Zielbuchse ziehen.
4. Loslassen, um die Verbindung herzustellen.

### 3.3.2. Leitung entfernen

Du hast mehrere Möglichkeiten:

- Dieselbe Verbindung erneut ziehen; die Richtung spielt keine Rolle.
- Kabel anklicken oder antippen, um es zu markieren, und erneut anklicken oder antippen.
- Ein markiertes Kabel mit **Entf** oder **Rücktaste** löschen.
- Ein Kabel per Rechtsklick entfernen.

### 3.3.3. Leitung umstecken

Markiere das Kabel, ziehe einen Stecker von seiner bisherigen Buchse weg und lasse ihn an einer neuen Buchse los. Wird der Stecker im Leeren losgelassen, entfernt der Simulator das Kabel.

### 3.3.4. Rückgängig

Mit **Strg+Z** unter Windows/Linux, **+Z** unter macOS oder der Schaltfläche  lassen sich bis zu 50 Kabeländerungen rückgängig machen. **Esc** hebt die Kabelmarkierung auf.



Kabel dürfen nur bei ausgeschaltetem DigiTrainer und außerhalb des Vermessungsmodus bearbeitet werden.

## 3.4. Spannungsversorgung und Verbindungsschienen

Die Logik-ICs benötigen eine korrekte Versorgung mit +5 V und GND. Die entsprechenden Versorgungsschienen sind innerhalb ihrer Gruppe elektrisch verbunden. Die fünf passiven Verbindungsschienen L0 bis L4 bieten jeweils vier miteinander verbundene Buchsen. Sie helfen dabei, ein Signal zu verteilen und lange Leitungen zu vermeiden.



Ein direktes Verbinden von +5 V und GND verursacht einen Kurzschluss. Der Simulator erkennt Kurzschlüsse und Ausgangskonflikte. In realen Schaltungen können solche Fehler Bauteile beschädigen.

## 3.5. Anzeige, Zoom und Touch

Der DigiTrainer unterstützt Zoomstufen von 100 % bis 400 %, Schaltflächen und Tastatur, Mausrad-Zoom, Zwei-Finger-Zoom sowie das Verschieben der vergrößerten Platine. Mit **0** kehrst du zu 100 % zurück; + und − verändern den Zoom. Vollbild wird auf PC und iPad unterstützt; auf dem iPhone steht ein Pseudo-Vollbildmodus zur Verfügung.

## 3.6. Vermessungsmodus

Mit **M** wechselst du zwischen Verdrahtungs- und Vermessungsmodus. Im Vermessungsmodus können Koordinaten und rechteckige Bereiche auf dem Platinenfoto ermittelt werden. Die Ausgabe im JSON-Format dient insbesondere dazu, weitere Steckbuchsen-Hotspots zu definieren.

## 3.7. Die Schaltplanansicht (neu in v0.69)

Über die Schaltfläche **Schaltplan** öffnest du eine automatisch erzeugte Schaltplanansicht der aktuell gesteckten Verdrahtung. Die Darstellung ordnet die verwendeten Gatter und Anschlüsse möglichst entlang des Signalflusses von links nach rechts an. +5 V und GND werden zur besseren Übersicht nicht eingezeichnet; das bedeutet nicht, dass die Bausteine ohne Versorgung funktionieren.

Die Schaltplanansicht bietet folgende Möglichkeiten:

- **IEC 60617:** Rechteckige Gattersymbole als Voreinstellung; alternativ können die herkömmlichen Gattersymbole gewählt werden.
- **Kabelfarben:** Auf Wunsch werden die Farben der gesteckten Verbindungen übernommen.
- **SVG speichern:** Der Schaltplan wird als skalierbare Vektorgrafik exportiert. Das ist besonders für Dokumentationen und dieses Handbuch nützlich.
- **Drucken:** Eine druckfreundliche Ansicht mit weißem Hintergrund ausgeben. Bei breiten Schaltungen empfiehlt sich Querformat.
- **Projektname:** In der Titelleiste erscheint der Name des geladenen Projekts ohne die Dateiendung `.json`, soweit ein Dateiname bekannt ist.



Die Anordnung und Leitungsführung erfolgen automatisch und können bei komplexen Verdrahtungen unübersichtliche Abschnitte enthalten. Bei größeren Schaltungen zeigt ein Fortschrittsbalken die Verarbeitung an. Bereits berechnete Ansichten werden zwischengespeichert. Die Schaltplanansicht stellt die vorhandene Verdrahtung dar; sie ersetzt keine Prüfung der logischen Funktion.

### 3.7.1. Ein Schaltbild für das Handbuch exportieren

1. Ein Beispielprojekt laden oder eine eigene Schaltung verdrahten.
2. Auf **Schaltplan** klicken.
3. Die gewünschte Symbolform und gegebenenfalls die Kabelfarben auswählen.
4. Den Schaltplan als SVG-Datei speichern.
5. Die SVG-Datei für die spätere Abbildung im Handbuch unter `docs/images/` ablegen.

**Weiterführende technische Dokumentation:** Schaltplan aus der Netzliste (PDF) (von Lutz Schulze, im Verzeichnis `docs/`).

# Kapitel 4. Teil II: Digitale Elektronik und simulierte Baugruppen

## 4.1. Grundlagen: LOW, HIGH und Wahrheitstabellen

Digitale Schaltungen arbeiten mit zwei logischen Zuständen: LOW (0) und HIGH (1). Ein Gatter berechnet aus seinen Eingangssignalen einen Ausgangszustand. Eine Wahrheitstabelle zeigt alle möglichen Eingangskombinationen.

### 4.1.1. NOT (Inverter)

| Eingang A | Ausgang Y |
|---|---|
| 0 | 1 |
| 1 | 0 |

### 4.1.2. AND

| A | B | Y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

### 4.1.3. OR

| A | B | Y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

### 4.1.4. XOR

| A | B | Y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

NAND ist die Negation von AND; NOR ist die Negation von OR. Der Simulator unterscheidet außerdem undefinierte Zustände. Offene Eingänge der simulierten 74LS-TTL-Gatter werden logisch als HIGH interpretiert; offene CMOS-Eingänge des CD4069 bleiben undefiniert.

## 4.2. Die sechs Logik-IC-Sockel

| Sockel | Baustein | Funktion |
|---|---|---|
| 0 | SN74LS02N | Vier NOR-Gatter mit je zwei Eingängen |
| 1 | SN74LS04N | Sechs Inverter (NOT) |
| 2 | SN74LS08N | Vier AND-Gatter mit je zwei Eingängen |
| 3 | SN74LS86AN | Vier XOR-Gatter mit je zwei Eingängen |
| 4 | 74LS00PC | Vier NAND-Gatter mit je zwei Eingängen |
| 5 | CD4069UBE | Sechs CMOS-Inverter |

Die simulierten Bausteine können miteinander verdrahtet werden. Für jedes Experiment müssen die Versorgungspins des verwendeten ICs korrekt angeschlossen werden. Die genaue Pinbelegung wird im Anhang nach Prüfung der Boarddefinition ergänzt.

## 4.3. Taktgenerator

Der integrierte Taktgenerator unterstützt 1 Hz und 10 Hz, manuelle Impulse, Start/Stopp sowie einen normalen und einen invertierten Taktausgang. Er ermöglicht Versuche mit Zählern und zeitabhängigen Schaltungen.

## 4.4. 4-Bit-Binärzähler SN74LS293N

Der Zähler besitzt die Takteingänge CK A und CK B, vier Ausgänge QA bis QD und zwei asynchrone Reset-Eingänge. In der üblichen Zählerbeschaltung liefert er eine Folge von 0 bis 15. Für eine vollständige 4-Bit-Zählfolge wird QA mit CK B verbunden, während CK A die Taktimpulse erhält.

## 4.5. 7-Segment-Anzeige MC14495P

Vier binäre Eingänge mit den Wertigkeiten 1, 2, 4 und 8 steuern die hexadezimale Darstellung von 0 bis F. Der Eingang E beeinflusst die Latch-Funktion. So kann der Zählerstand direkt sichtbar gemacht werden.

## 4.6. 4-Bit-ALU SN74LS382N

Die ALU verarbeitet zwei 4-Bit-Operanden A0–A3 und B0–B3. Die Steuereingänge S0–S2 wählen eine Funktion; Cn ist der Eingangsübertrag. Ausgänge sind F0–F3, OVR und Cn+4. Der Simulator unterstützt unter anderem Addition, Subtraktion und logische Operationen. Ein dokumentierter Funktionstest ist 3 + 2 = 5.

## 4.7. 6-Bit-A/D-Wandler NE5037N

Der NE5037N ist als **experimentelles, vereinfachtes Modell** implementiert. Der Wandler liefert sechs digitale Ausgänge Q0–Q5. Als geprüfte Grenzwerte sind 0 V → `000000` und +5 V → `111111` dokumentiert. Zwischenwerte und zeitliche Details sind noch nicht abschließend verifiziert.

# Kapitel 5. Teil III: Experimente

## 5.1. So sind die Experimente aufgebaut

Jedes Experiment enthält Lernziel, benötigte Funktionsgruppen, geprüfte Verdrahtung, Durchführung, erwartete Beobachtung, Erklärung und eine Zusatzaufgabe. Die Stecklisten werden aus der Boarddefinition und gespeicherten JSON-Projekten übernommen und anschließend im Simulator getestet.

## 5.2. Experiment 1 – Schalter, LED und NOT-Gatter

In unserem ersten Experiment lernen wir zwei Grundbausteine digitaler Schaltungen kennen: einen Schalter als Eingang und eine Leuchtdiode (LED) als Ausgang. Anschließend schalten wir ein NOT-Gatter dazwischen und beobachten, wie sich das Verhalten verändert.

### 5.2.1. Lernziele

Nach diesem Experiment kannst du:

- die logischen Zustände `0` (LOW) und `1` (HIGH) unterscheiden,
- den Zustand eines Schalters mit einer LED sichtbar machen,
- die Funktion eines NOT-Gatters erklären,
- ein Gatter des 74LS04 auf dem DigiTrainer verdrahten und
- die zugehörige Wahrheitstabelle lesen.

### 5.2.2. Vorbereitung

Öffne den DigiTrainer im Browser. Zum Verdrahten muss der Hauptschalter zunächst auf **OFF** stehen. Schalte erst nach dem Verbinden der Buchsen auf **ON**, um die Schaltung zu testen.

Für die folgenden Versuche verwenden wir den Schalter **S0** und die LED **D0**. Alle anderen Schalter bleiben unbenutzt.

### 5.2.3. Experiment 1a – Schalter und LED

#### Aufgabe

Verbinde den Schalter S0 direkt mit der LED D0. So lässt sich der logische Zustand des Schalters unmittelbar anzeigen.

#### Verdrahtung

| Von | Nach |
|---|---|
| S0 | D0 |




**Durchführung**

1. Stelle den Hauptschalter auf **OFF**.
2. Verbinde die Buchse **S0** mit der Buchse **D0**.
3. Schalte den DigiTrainer auf **ON**.
4. Betätige S0 mehrmals und beobachte D0.

**Beobachtung**

| S0 | Logischer Zustand | D0 |
|---|---|---|
| OFF | 0 (LOW) | Aus |
| ON | 1 (HIGH) | Ein |

Die LED folgt dem Schalter: Bei S0 = 0 bleibt sie aus; bei S0 = 1 leuchtet sie.

**Projektdatei:** experiment01a-schalter-led.json

### 5.2.4. Experiment 1b – Das NOT-Gatter (Inverter)

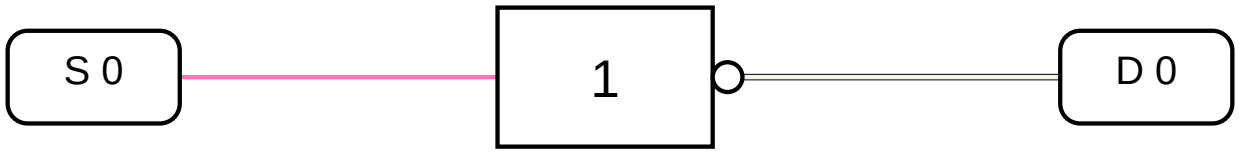
**Aufgabe**

Nun schalten wir zwischen S0 und D0 ein NOT-Gatter. Es kehrt den logischen Zustand um: Aus einer `0` wird eine `1` und umgekehrt.

Dafür verwenden wir ein Gatter des **74LS04** in IC-Sockel 1 des DigiTrainers.

**Verdrahtung**

| Von | Nach | Bedeutung |
|---|---|---|
| +5 V (Buchse VCC4) | IC1, Buchse 16 | Versorgungsspannung |
| GND (Buchse GND3) | IC1, Buchse 7 | Masse |
| S0 | IC1, Buchse 1 | Eingang des NOT-Gatters |
| IC1, Buchse 2 | D0 | Ausgang des NOT-Gatters |




**Besonderheit des DigiTrainers:** Der 74LS04 ist ein 14-poliger Baustein, wird hier aber in einem 16-poligen IC-Sockel verwendet. Deshalb ist zwischen den **Pin-Nummern des ICs** und den **Buchsennummern des DigiTrainers** zu unterscheiden. Für die hier verwendete Bestückung liegt die Versorgung des 74LS04 (IC-Pin 14) an **Sockelbuchse 16**; Masse (IC-Pin 7) liegt an **Sockelbuchse 7**. Eingang und Ausgang des ersten Inverters sind über die Buchsen 1 und 2 erreichbar.

**Durchführung**

1. Schalte den DigiTrainer auf **OFF**.
2. Entferne die direkte Verbindung von S0 nach D0 aus Experiment 1a.
3. Verbinde die vier Buchsenpaare entsprechend der Verdrahtungstabelle.
4. Schalte den DigiTrainer auf **ON**.
5. Betätige S0 mehrmals und beobachte D0.

**Wahrheitstabelle**

| S0 (Eingang) | Ausgang des NOT-Gatters | D0 |
|---|---|---|
| 0 (LOW) | 1 (HIGH) | Ein |
| 1 (HIGH) | 0 (LOW) | Aus |

Anders als bei Experiment 1a zeigt D0 nun stets den **entgegengesetzten** Zustand des Schalters S0 an. Dieses Verhalten nennt man **Invertierung**.

Die logische Funktion schreiben wir als:

$$Y = \neg A$$

Dabei steht `A` für den Eingang und `Y` für den invertierten Ausgang.

**Projektdatei:** experiment01b-not-gatter.json

## 5.2.5. Vergleich der beiden Versuche

| Schalter S0 | D0 bei 1a (direkt) | D0 bei 1b (NOT) |
|---|---|---|
| 0 | Aus | Ein |
| 1 | Ein | Aus |

Das NOT-Gatter verändert also nicht die Anzahl der möglichen Zustände, sondern vertauscht sie.

### 5.2.6. Fragen und Übungen

1. Was zeigt die LED D0 in Experiment 1a an, wenn S0 eingeschaltet ist?
2. Warum leuchtet D0 in Experiment 1b, wenn S0 ausgeschaltet ist?
3. Welche Funktion hat die Verbindung zur Versorgungsspannung (+5 V) und zur Masse (GND)?
4. Was würde sich ändern, wenn du statt S0 den Schalter S1 verwenden möchtest? Welche Verbindung müsstest du umstecken?
5. Vergleiche die beiden Wahrheitstabellen. Welche Zeile ändert sich durch das NOT-Gatter?

### 5.2.7. Merksatz

Ein **NOT-Gatter** hat einen Eingang und einen Ausgang. Es liefert immer den entgegengesetzten logischen Zustand des Eingangs: `NOT 0 = 1` und `NOT 1 = 0`.

## 5.3. Experiment 2: Ein 4-Bit-Zähler zeigt Hexadezimalzahlen

**Lernziel:** Binäre Zählwerte und ihre hexadezimale Darstellung kennenlernen.

**Vorlage:** `projects/counter-1.json`

1. DigiTrainer öffnen und auf **Projekt** klicken.
2. Unter **Beispielprojekte** `counter-1.json` auswählen und die Beschreibung lesen.
3. Das Projekt laden.
4. Prüfen, dass der DigiTrainer zunächst ausgeschaltet ist.
5. Den Hauptschalter einschalten und den Taktgenerator starten.
6. Die Anzeige von 0 bis F beobachten.

**Erklärung:** Vier Binärstellen können 16 Werte darstellen. Die 7-Segment-Anzeige übersetzt diese Werte in die Ziffern 0 bis 9 und die Buchstaben A bis F.

**Zusatzaufgabe:** Zwischen 1 Hz und 10 Hz wechseln. Bei welcher Frequenz lassen sich die einzelnen Zählerstände besser verfolgen? Öffne außerdem die **Schaltplanansicht** und speichere die Darstellung als SVG.

## 5.4. Experiment 3: Binärzähler und LED-Ausgänge

**Lernziel:** Die Bitwertigkeiten 1, 2, 4 und 8 an den Ausgängen beobachten.

**Vorlage:** `projects/counter-2.json`

Lade die Beispielschaltung und starte den Taktgenerator. Beobachte die 7-Segment-Anzeige und die zusätzlichen binären Ausgänge D4–D7. Untersuche anschließend, welchen Einfluss S0 auf den

Anzeigeeingang E hat.

## 5.5. Experiment 4: Lauflicht D0 bis D7

**Lernziel:** Aus einem Zähler und kombinatorischer Logik eine zeitlich fortschreitende LED-Folge erzeugen.

**Vorlage:** `projects/lauflicht-D0-D7.json`

Lade die Schaltung aus der Projektbibliothek. Schalte den DigiTrainer ein und starte den Taktgenerator. Beobachte, wie die aktive LED von D0 nach D7 wandert. Die Beispielschaltung verwendet vorhandene Logik-ICs und den integrierten Zähler. Vergleiche anschließend die Verdrahtung mit der automatisch erzeugten Schaltplanansicht.

## 5.6. Experiment 5: Pendellauflicht

**Lernziel:** Eine vorwärts- und rückwärtslaufende LED-Folge verstehen.

**Vorlage:** `projects/lauflicht-D0-D7-Pendel.json`

Lade das Pendellauflicht, schalte ein und starte den Takt. Beobachte die Folge von D0 nach D7 und zurück. Achte besonders auf die doppelte Leuchtdauer an den Endpunkten. Betrachte anschließend die Schaltplanansicht in beiden Symbolformen.

# Kapitel 6. Teil IV: Projekte, Fehlersuche und Technik

## 6.1. Projekte speichern und laden

Der Button **Projekt** öffnet die Projektverwaltung. **Meine Projekte** enthält im Browser gespeicherte Schaltungen. **Beispielprojekte** zeigt die bereitgestellten JSON-Dateien aus dem GitHub-Repository. Für diese öffentliche Bibliothek ist eine Internetverbindung erforderlich.

Du kannst Projekte lokal im Browser speichern, als JSON exportieren und JSON-Dateien importieren. Der Export ist die zuverlässigere Sicherung, da Browserdaten gelöscht werden können. Gespeichert werden Verbindungen, Schalterstellungen, Taktgenerator-Einstellungen, Debug-Zustand und gegebenenfalls eine Beschreibung. Interne Laufzeitzustände, etwa ein aktueller Zählerstand, werden nicht mitgespeichert. Beim Laden ist der DigiTrainer ausgeschaltet.

## 6.2. Beschreibungen in JSON

Eine Projektbeschreibung steht vorzugsweise direkt im Feld `description`:

```
{
  "format": "digitrainer-projekt",
  "version": 1,
  "description": "4-Bit-Binärzähler mit 7-Segment-Anzeige.\n\nNach dem Laden
einschalten und den Taktgenerator starten."
}
```

Dieses Beispiel zeigt nur die Metadaten und ist **keine vollständig importierbare Schaltung**. In echten Projektdateien kommen unter anderem `board`, `wires` und `switches` hinzu. `\n` erzeugt einen Zeilenumbruch; `\n\n` eine Leerzeile. Fehlt `description`, kann die Bibliothek alternativ eine gleichnamige `.txt`-Datei lesen. Die JSON-Beschreibung hat Vorrang.

## 6.3. Eigene Projekte beitragen

Die Beispielbibliothek soll durch Schaltungen aus der Community wachsen. Erstelle eine Schaltung, teste sie im Simulator und exportiere sie als JSON-Datei. Ergänze eine kurze Beschreibung mit Zweck und Bedienhinweisen und sende die Datei an den Projektbetreiber. Nach Prüfung kann sie in die öffentliche Bibliothek aufgenommen werden.

## 6.4. Debug-Anzeige

Bei eingeschaltetem Gerät zeigt **Debug** die logischen Pegel: Rot für HIGH (1), Hellblau für LOW (0) und Hellgrau für undefiniert. Zusätzlich werden binäre Werte unter Schaltern und LEDs angezeigt. Ohne Debug werden zusammenhängende elektrische Netze durch gleiche Kabelfarben kenntlich gemacht; Versorgung und Masse erhalten eigene Farben.

## 6.5. Typische Fehler

| Beobachtung | Prüfung |
|---|---|
| Ein Kabel lässt sich nicht stecken | Hauptschalter OFF? Vermessungsmodus deaktiviert? |
| Ein IC reagiert nicht | +5 V und GND korrekt verbunden? Richtige Ein- und Ausgänge verwendet? |
| Ein Signal bleibt undefiniert | Offener CMOS-Eingang oder nicht versorgter Baustein? |
| Ausgangskonflikt | Sind zwei Ausgänge direkt miteinander verbunden? |
| Beispielprojekt fehlt | Internetverbindung und Zugriff auf die GitHub-Bibliothek prüfen. |
| Schaltplan braucht länger | Bei komplexen Projekten Fortschrittsanzeige abwarten. |
| Schaltplan wirkt unübersichtlich | Alternative Symbolform oder Kabelfarben testen; SVG exportieren und vergrößern. |
| Projekt nach Browserbereinigung verschwunden | Nach Möglichkeit aus einer zuvor exportierten JSON-Datei importieren. |

## 6.6. Technische Umsetzung und Grenzen

Der DigiTrainer verwendet HTML, CSS und JavaScript. Die Steckbuchsen werden anhand einer Boarddefinition zugeordnet; Patchkabel bilden elektrische Netze. Der Simulator berechnet daraus die logischen Zustände. Er ersetzt keine vollständige elektrische Schaltungssimulation: reale Laufzeiten, Stromaufnahme, Belastbarkeit und analoge Grenzfälle sind nur eingeschränkt modelliert. Der A/D-Wandler befindet sich ausdrücklich in einem experimentellen Entwicklungsstadium.

Für den lokalen Betrieb empfiehlt sich ein Webserver im Repository-Verzeichnis, beispielsweise:

```
python3 -m http.server 8000
```

Anschließend `http://localhost:8000/` öffnen. Das direkte Öffnen von `index.html` über `file://` kann durch Browser-Sicherheitsbeschränkungen das Laden der Boarddefinition verhindern. Die eigentliche Simulation benötigt keine Internetverbindung; die online bereitgestellten Beispielprojekte hingegen schon.

## 6.7. Wie entsteht der automatische Schaltplan?

Die Schaltplanerstellung wurde von Lutz (`lschulze`) in Pull Request #4 beigesteuert. Sie wertet die vorhandenen elektrischen Netze aus, erkennt die daran angeschlossenen Logikfunktionen und erstellt daraus eine grafische Darstellung. Für die Anordnung der Elemente werden Verfahren aus der Graphendarstellung verwendet; die Leitungen werden anschließend mit möglichst gut lesbaren Abständen und Verzweigungen geführt.

Die Schaltplanansicht ist von der eigentlichen Simulation getrennt: Die Einführung erfolgte ohne Änderungen an `js/simulator.js`. Die Umsetzung befindet sich in `js/schaltplan.js`; `js/projekt.js` stellt unter anderem den geladenen Projektnamen bereit. Ein ergänzender Test unter `test/leitungsabstand.js` untersucht die Leitungsführung anhand der Beispielprojekte.

Für die geplante austauschbare IC-Bibliothek muss die Schaltplanerstellung später auch die Definitionen neu ausgewählter ICs berücksichtigen. Die technische Beschreibung von Lutz erläutert die derzeitige Vorgehensweise ausführlicher: Schaltplan aus der Netzliste.

# Anhang A: Funktions- und Quellenprüfung

| Bereich | Quelle für die Endredaktion | Status |
|---|---|---|
| Bedienung, Kabel, Undo, Zoom | README und `js/kabel.js` | README dokumentiert; Codeabgleich offen |
| ICs, Zähler, ALU, ADC | README, `js/simulator.js`, `board.json` | README dokumentiert; Codeabgleich offen |
| Projektverwaltung | README und `js/projekt.js` | README dokumentiert; Codeabgleich offen |
| Stecklisten der Experimente | `board.json` und vier Projekt-JSONs | Stecklistenprüfung offen |
| Schaltplanansicht und SVG-Export | `js/schaltplan.js`, PR #4, technische PDF-Dokumentation | Funktionen dokumentiert; Abgleich mit Quellcode offen |
| Bildschirmfotos | Live-Demo v0.69 | Bildauswahl offen |

# Anhang B: Quellen und Mitwirkende

- DigiTrainer: https://github.com/ManiBecker/DigiTrainer
- Live-Demo: https://manibecker.github.io/DigiTrainer/
- Projekt-README (Stand der Arbeitsfassung): https://raw.githubusercontent.com/ManiBecker/DigiTrainer/main/README.md

Der Simulator basiert auf Fotografien eines originalen ITT Digi-Trainer 2, die jobi23 aus dem Classic-Computing-Forum bereitgestellt hat. Lutz (`lschulze`) hat unter anderem zur Projektverwaltung, Kabelbedienung, Projektbibliothek und zur automatischen Schaltplanerstellung (Pull Request #4) beigetragen. Seine technische Dokumentation liegt unter Schaltplan aus der Netzliste.

# Anhang C: IC-Katalog für künftige Experimente

Dieser Katalog sammelt Bausteine, die sich für spätere DigiTrainer-Experimente und eine austauschbare IC-Bibliothek eignen. **Die Aufnahme in diese Liste bedeutet nicht, dass der Baustein bereits simuliert oder in einem Sockel auswählbar ist.** Die Verfügbarkeit und die Pinbelegung müssen vor jedem Experiment geprüft werden.

## C.1. Bereits simulierte ICs

| Baustein | Pins | Funktion | Experimentideen |
|---|---|---|---|
| 74LS00 | 14 | Vier NAND-Gatter mit je zwei Eingängen | NAND-Grundschaltungen, RS-Speicher |
| 74LS02 | 14 | Vier NOR-Gatter mit je zwei Eingängen | NOR-Grundschaltungen, RS-Speicher |
| 74LS04 | 14 | Sechs Inverter | Signalumkehr, einfache Logik |
| 74LS08 | 14 | Vier AND-Gatter mit je zwei Eingängen | Freigabeschaltungen, Halbaddierer |
| 74LS86 | 14 | Vier XOR-Gatter mit je zwei Eingängen | Halbaddierer, Paritätsprüfung |
| CD4069UBE | 14 | Sechs CMOS-Inverter | Vergleich von TTL- und CMOS-Eingangsverhalten |

Der **CD4069UBE ist kein TTL-IC**, sondern ein CMOS-Baustein der 4000er-Familie. Er ist hier aufgeführt, weil er zur vorhandenen Bestückung gehört. Zusätzlich sind auf dem Board unter anderem der Binärzähler SN74LS293N, die 7-Segment-Ansteuerung MC14495P, die ALU SN74LS382N und der experimentell modellierte A/D-Wandler NE5037N vorhanden. Diese Baugruppen sind nicht mit den sechs frei zu bestückenden IC-Plätzen gleichzusetzen.

## C.2. Logikgatter und kombinatorische Funktionen

| Baustein | Pins | Funktion | Experimentideen |
|---|---|---|---|
| 74LS10 | 14 | Drei NAND-Gatter mit je drei Eingängen | Mehrfachbedingungen |
| 74LS20 | 14 | Zwei NAND-Gatter mit je vier Eingängen | Freigabelogik |
| 74LS21 | 14 | Zwei AND-Gatter mit je vier Eingängen | Zustandsabfragen |
| 74LS27 | 14 | Drei NOR-Gatter mit je drei Eingängen | Kombinatorische Logik |

| Baustein | Pins | Funktion | Experimentideen |
|---|---|---|---|
| 74LS30 | 14 | Ein NAND-Gatter mit acht Eingängen | Bitmuster erkennen |
| 74LS32 | 14 | Vier OR-Gatter mit je zwei Eingängen | OR-Verknüpfungen |
| 74LS266 | 14 | Vier XNOR-Gatter mit Open-Collector-Ausgängen | Gleichheitsprüfung, Ausgangsbeschaltung |

## C.3. Flipflops und Schieberegister

| Baustein | Pins | Funktion | Experimentideen |
|---|---|---|---|
| 74LS74 | 14 | Zwei D-Flipflops | Speichern, Frequenzteilen |
| 74LS76 | 16 | Zwei JK-Flipflops | Toggle-Betrieb, Zähler |
| 74LS109 | 16 | Zwei JK-Flipflops | Zustandsautomaten |
| 74LS164 | 14 | 8-Bit-Schieberegister, seriell nach parallel | LED-Lauflicht |
| 74LS165 | 16 | 8-Bit-Schieberegister, parallel nach seriell | Serielle Datenausgabe |
| 74LS173 | 16 | 4-Bit-Register mit Tri-State-Ausgängen | Datenspeicherung, Bus |
| 74LS175 | 16 | Vier D-Flipflops mit gemeinsamem Takt | 4-Bit-Register |
| 74LS194 | 16 | Bidirektionales 4-Bit-Schieberegister | Verschieben nach links und rechts |

## C.4. Zähler und Frequenzteiler

| Baustein | Pins | Funktion | Experimentideen |
|---|---|---|---|
| 74LS90 | 14 | Dezimalzähler | Zählen von 0 bis 9 |
| 74LS92 | 14 | Modulo-12-Zähler | Frequenzteilung |
| 74LS93 | 14 | 4-Bit-Binärzähler | Zählen von 0 bis 15 |
| 74LS161 | 16 | Synchroner 4-Bit-Binärzähler | Synchrones Zählen |
| 74LS163 | 16 | Synchroner 4-Bit-Binärzähler | Kaskadierte Zähler |
| 74LS192 | 16 | Synchroner BCD-Auf-/Abwärtszähler | Dezimal vorwärts und rückwärts zählen |
| 74LS193 | 16 | Synchroner 4-Bit-Auf-/Abwärtszähler | Pendelzähler |

## C.5. Decoder, Multiplexer und Anzeige

| Baustein | Pins | Funktion | Experimentideen |
|---|---|---|---|
| 74LS47 | 16 | BCD-zu-7-Segment-Decoder/Treiber | Dezimalanzeige mit geeigneter Anzeige |
| 74LS138 | 16 | 3-zu-8-Decoder | LED-Auswahl, Adressdekodierung |
| 74LS139 | 16 | Zwei 2-zu-4-Decoder | Adressdekodierung |
| 74LS148 | 16 | 8-zu-3-Prioritätsencoder | Prioritäten auswerten |
| 74LS151 | 16 | 8-zu-1-Multiplexer | Datenauswahl |
| 74LS153 | 16 | Zwei 4-zu-1-Multiplexer | Logikfunktionen auswählen |
| 74LS157 | 16 | Vier 2-zu-1-Multiplexer | 4-Bit-Datenquellen umschalten |

Der 74LS47 ist für Anzeigen mit passender elektrischer Ansteuerung gedacht. Er kann nicht ohne Weiteres als Ersatz für den auf dem DigiTrainer vorhandenen MC14495P betrachtet werden.

## C.6. Arithmetik und Vergleicher

| Baustein | Pins | Funktion | Experimentideen |
|---|---|---|---|
| 74LS83 | 16 | 4-Bit-Volladdierer | Binäre Addition |
| 74LS85 | 16 | 4-Bit-Größenkomparator | A = B, A größer/kleiner B |
| 74LS181 | 24 | 4-Bit-ALU | Kleines Rechenwerk; zusätzlicher Sockel nötig |
| 74LS283 | 16 | 4-Bit-Volladdierer | Addition und Subtraktion mit Zusatzlogik |

## C.7. Besonders interessante 20-polige Bausteine

| Baustein | Pins | Funktion | Experimentideen |
|---|---|---|---|
| 74LS240 | 20 | Acht invertierende Tri-State-Bustreiber | Busfreigabe, Datenbus |
| 74LS244 | 20 | Acht nichtinvertierende Tri-State-Bustreiber | Busfreigabe |
| 74LS245 | 20 | Bidirektionaler 8-Bit-Bustransceiver | Datenübertragung in beide Richtungen |
| 74LS273 | 20 | Acht D-Flipflops mit gemeinsamem Takt | 8-Bit-Datenspeicher |
| 74LS373 | 20 | Acht transparente Latches mit Tri-State-Ausgängen | Zwischenspeichern von Busdaten |

| Baustein | Pins | Funktion | Experimentideen |
|---|---|---|---|
| 74LS374 | 20 | Acht flankengesteuerte D-Flipflops mit Tri-State-Ausgängen | 8-Bit-Register und Datenbus |

## C.8. Sockel und elektrische Besonderheiten

- Die **Sockel 0 bis 4 sind 16-polig**, **Sockel 5 ist 20-polig**. Das muss die spätere Bausteinauswahl berücksichtigen.
- Die vielen 14-poligen TTL-ICs benötigen eine definierte Zuordnung zu den 16-poligen Steckplätzen. Eine physisch und elektrisch korrekte Einsetzposition darf nicht stillschweigend angenommen werden.
- 20-polige ICs sind Kandidaten für Sockel 5. Der 24-polige 74LS181 passt dagegen in keinen der vorhandenen Sockel.
- Stromversorgung, Pinbelegung, Reset-/Takteingänge und Ausgangstypen müssen für jeden IC individuell modelliert werden.
- Open-Collector-Ausgänge benötigen eine geeignete Pull-up-Beschaltung; Tri-State-Ausgänge können hochohmig sein. Solche Eigenschaften müssen vom Simulator unterstützt werden, bevor entsprechende Experimente als funktionsfähig gelten.
- Nicht alle Bausteine der 74xx-Familie sind untereinander elektrisch oder zeitlich identisch. Für die Dokumentation ist die konkrete Typbezeichnung maßgeblich.

## C.9. Vorgeschlagene Reihenfolge für die IC-Bibliothek

| Priorität | IC | Grund |
|---|---|---|
| 1 | 74LS74 | Einfacher Einstieg in speichernde Logik |
| 2 | 74LS32 | Ergänzt die Grundgatter um OR |
| 3 | 74LS193 | Auf- und Abwärtszähler |
| 4 | 74LS164 | Serielle Daten und LED-Lauflichter |
| 5 | 74LS138 | Decoder und gezielte Ausgangsauswahl |
| 6 | 74LS151 | Multiplexer und Datenauswahl |
| 7 | 74LS85 | Vergleich zweier Binärzahlen |
| 8 | 74LS283 | Vier-Bit-Addition |
| 9 | 74LS273 | Nutzt den 20-poligen Sockel 5 für ein 8-Bit-Register |



Dieser Anhang ist ein **Planungskatalog**, keine Liste bereits unterstützter Austausch-ICs. Pinzahlen und Funktionen sind zur Orientierung angegeben; konkrete Pinbelegungen, Versorgungsanforderungen und Simulationseigenschaften müssen vor der Implementierung mit den jeweiligen Datenblättern abgeglichen werden.